

MP1 Report — Rotary Position Encoding and Model Width

DASE7506 Small Language Model Challenge · 29 September 2026

1. Summary

This project trains a causal language model from random initialization on the supplied WikiText-2 training text. The course baseline uses learned absolute position embeddings in a four-block GPT with width 128. I first replaced those embeddings with rotary position encoding (RoPE) in attention, then increased the RoPE model width from 128 to 192. The final selected model was trained for 4,800 updates and obtained **1.642035 validation BPB** and **1.665363 full-test BPB** with FP32 CPU scoring. The initial baseline obtained 2.071088 validation BPB and 2.101266 full-test BPB. Lower BPB is better.

The two design choices have separate, equal-target comparisons: baseline versus RoPE at 1,200 updates, and width 128 versus width 192 with RoPE at 4,800 updates. These experiments show improvements in this setting, while also showing increased training and scoring cost. They do not establish that either choice is universally better.

2. Task and controlled setup

The fixed protocol is 7506-mp1-wt2-v2: WikiText-2 raw text, a supplied training-fitted BPE tokenizer with vocabulary 2,048, and independent causal windows of 256 targets. The metric is summed next-token negative log probability in bits divided by the raw UTF-8 byte count. The validation split has 376,599 scored targets and 1,148,007 bytes; the test split has 428,405 scored targets and 1,292,013 bytes. The data, tokenizer, `common.py`, and `evaluate.py` were not changed.

All five recorded training runs started from random initialization with seed 17. They used the supplied `train.py` recipe: batch size 32, AdamW, the existing warm-up and cosine learning-rate schedule, weight decay 0.1, context 256, and CPU FP32 with four threads. Validation BPB was used to choose the architecture and training length. The final checkpoint records the student implementation and width-192 configuration; it does not require retraining for evaluation.

3. Method and reasons for choosing it

3.1 Rotary position encoding

The baseline adds a learned absolute position vector to each token embedding. My hypothesis was that a positional rule applied inside attention could represent relative offsets more directly within the independent 256-token windows. Following the RoPE idea of Su et al. [1], `student.py` rotates paired dimensions of each attention query and key by a position-dependent angle before causal scaled-dot-product attention. Values, the four-block depth, the four attention heads, the feed-forward network, tied token/output embeddings, optimizer, and training data remain as in the baseline. The learned position-embedding table is removed. This reduces the width-128 parameter count from 1,088,256 to 1,055,488.

RoPE was chosen as a focused first change because it has a clear ablation: turn the positional mechanism back into the supplied baseline while keeping the training target count fixed. The course's causal-window tests passed after the change. The measured improvement is evidence for this positional design in this particular setup; because the learned position table is also removed, the comparison does not isolate rotation from the small parameter-count reduction.

3.2 Increasing model width

With RoPE, validation BPB continued to fall as training increased from 1,200 to 2,400 to 4,800 updates, but the gains became smaller. I tested whether additional representational capacity would help at a fixed 4,800-update budget. The only planned model change was width 128 to 192, keeping depth 4, heads 4, context 256, RoPE, and the training recipe unchanged. Parameter count rose from 1,055,488 to 2,173,056. This experiment tests a quality-versus-compute trade-off, not a new positional mechanism.

4. Results and ablations

All times below are local CPU training times, excluding setup and validation scoring. “Processed targets” includes repeated samples drawn by the trainer; it is not a count of unique text tokens. A dash means the run was not scored on the test split.

Model	Width	Updates	Processed targets	Parameters	Train time	Validation BPB	Test BPB
Course baseline, learned positions	128	1,200	9,830,400	1,088,256	354 s	2.071088	2.101266
RoPE	128	1,200	9,830,400	1,055,488	407 s	1.922326	—
RoPE	128	2,400	19,660,800	1,055,488	716 s	1.778574	—
RoPE	128	4,800	39,321,600	1,055,488	1,529 s	1.692997	1.719331
RoPE	192	4,800	39,321,600	2,173,056	2,246 s	1.642035	1.665363

At equal 1,200-update and 9,830,400-target budgets, the RoPE positional variant improves validation BPB by **0.148762** relative to the original position-embedding model. This is the key-mechanism ablation and the initial equal-target control. At equal 4,800-update and 39,321,600-target budgets, width 192 improves validation BPB by **0.050961** relative to width 128, while taking about **1.47 times** as long to train locally. The final test BPB is 0.435903 below the initial baseline test BPB. Training-length gains are not equal-compute architecture comparisons and are reported separately.

The 4,800-update width-192 run had validation BPB 1.788224, 1.690345, 1.651981, and 1.642035 at steps 1,200, 2,400, 3,600, and 4,800, respectively. This progression supported completion of the planned run, while showing diminishing late-stage improvement. The width-128 run showed the same pattern. No claim of an optimal training length follows from these four checkpoints.

5. Practical issues, costs, and limitations

The first setup attempt failed because this Mac had system Python 3.9 rather than the required Python 3.12. After installing Python 3.12 and dependencies, the supplied five contract tests passed. Training was CPU-only and slower than the course's reference Xeon timing. Validation and test scores must not be confused: 1.642035 is the final validation BPB, while **1.665363** is the score eligible for the leaderboard.

The five full training runs together used **117,964,800 processed training targets** and about **5,252 seconds (87.5 minutes)** of measured training time, before setup and evaluation overhead. All runs were independent; none reused a checkpoint, so there is no checkpoint-training ancestry to add. This is the recorded model-search cost, not just the cost of the winning run.

For the frozen width-192 model, a repeated full-test FP32 CPU evaluation took about **9.44 seconds**, versus **6.68 seconds** for the local baseline (about 1.41 times as long, below the 5-times limit). Peak resident memory measured during the full-test run was about **1.42 GiB**, below 4 GiB. The checkpoint is **8,709,749 bytes (8.31 MiB)**, with no extra model inference assets, below 64 MiB. The exact values are machine-dependent; the BPB is the reproducible assessment result.

The study uses one seed and one dataset. A stronger claim about robustness would need more seeds. The positional ablation changes both the RoPE computation and the presence of the learned position table. The width comparison changes parameter count and compute, so its gain is not free. The width-128 interim model had already been evaluated on the public test split and submitted before the width-192 experiment. The later width choice was made using validation BPB, but this sequence means the final test set was not wholly unseen throughout the project. No validation or test text, per-window answer, or external text was used for training or retrieval, and no evaluator, tokenizer, or data file was modified.

6. Reproduction and disclosure

Exact installation, training, and evaluation commands are in `README.md`. The final checkpoint SHA-256 is `aa35c0455a95d5cc4cbe124b4e57ec257b96012b056160a60c784764ec03bffa1`; the matching `student.py` SHA-256 is `a90a2d9ebd047e791425e350ce9a226bf5a9a1ed3b3e29227d0893ae94f0e328`. The checkpoint, this implementation, `configs/rope_width192.json`, the unchanged evaluator, data, and tokenizer are needed for direct reproduction.

The baseline model, trainer, evaluation pipeline, tokenizer, and data were provided by the course. The RoPE method was introduced in [1]; this project adapts that idea to the supplied baseline rather than claiming the method as original. **OpenAI Codex provided substantive assistance** in designing and implementing the RoPE variant, proposing and running the width comparison, and checking results and resource use. It also assisted with writing this report and the README additions.

References

- [1] Jianlin Su et al. "RoFormer: Enhanced Transformer with Rotary Position Embedding." [arXiv:2104.09864](https://arxiv.org/abs/2104.09864).
- [2] Stephen Merity et al. "Pointer Sentinel Mixture Models." [arXiv:1609.07843](https://arxiv.org/abs/1609.07843). Introduced the WikiText corpus used by the course benchmark.